Category 4: Databases 🗄️

While you could install PostgreSQL on a raw EC2 instance, you'd be responsible for patching, backups, and high availability. AWS Managed Databases handle the undifferentiated heavy lifting for you.

🐘 Amazon RDS (Relational Database Service) MUST KNOW

What it is: A managed service that makes it easy to set up, operate, and scale a relational database in the cloud.

Analogy: Hiring a dedicated database administrator (DBA) who works 24/7. You still choose the database type, but they handle the backups, patching, and failing-over if a server dies.

Why and when to use it: When you need complex transactions, JOINs, and strict schema enforcement. Supports PostgreSQL, MySQL, MariaDB, Oracle, and SQL Server.

Key concepts:

- **Multi-AZ:** Synchronous replication to a standby instance in another Availability Zone for disaster recovery (High Availability).
- **Read Replicas:** Asynchronous copies used to scale out *read* heavy workloads (Performance).

Common example: The primary transactional database for an e-commerce platform (users, orders, payments).

Pricing model: Billed per hour based on instance size (e.g., `db.t3.medium`), plus allocated EBS storage, and data transfer.

Best practices and pitfalls: *Pitfall:* Using a Multi-AZ standby instance to run read queries. You cannot read from a Multi-AZ standby! You must create a Read Replica if you want to offload read traffic.

Note: 🎯 Exam Focus: Know the difference between Multi-AZ (Disaster Recovery) and Read Replicas (Performance scaling).

Related services: VPC (RDS must be in private subnets), KMS (for storage encryption).

🌅 Amazon Aurora MUST KNOW

What it is: A fully managed, MySQL- and PostgreSQL-compatible relational database built explicitly for the cloud by AWS.

⚡ Amazon DynamoDB MUST KNOW

What it is: A fast, flexible, fully managed NoSQL key-value and document database that delivers single-digit millisecond performance at any scale.

Analogy: A massive, infinitely scaling spreadsheet where you look up rows by a specific ID (Primary Key) instantly, but you can't easily join sheets together.

Why and when to use it: Web scale applications, user session data, gaming leaderboards, IoT sensor data. Perfect for Serverless (Lambda) architectures.

Key concepts: Partition Key (how data is distributed), Sort Key (how data is ordered), RCUs/WCUs (Read/Write Capacity Units - how you provision speed).

How it works: You do not provision servers. You create a table, set a primary key, and start throwing JSON documents at it via API calls.

Pricing model: Provisioned (pay for reserved RCUs/WCUs) or On-Demand (pay per request). Highly cost-effective for spiky workloads.

Note: 🎯 Exam Focus: If you see "single-digit millisecond latency" or "key-value store" in a requirement, the answer is almost always DynamoDB.

⚖️ RDS vs. DynamoDB

Feature	RDS / Aurora (Relational)	DynamoDB (NoSQL)
Data Structure	Strict Schema (Tables, Columns, Rows)	Flexible Schema (JSON Documents, Key-Value)
Querying	Complex SQL, multi-table JOINs	Simple lookups by Primary Key. No JOINs.
Scaling	Vertical scaling (bigger instance size)	Infinite horizontal scaling (automatic)
Maintenance	Maintenance windows, OS patching (managed)	Fully serverless, zero maintenance
Best For	ERP systems, financial ledgers, complex data	Session state, IoT, high-velocity simple reads

🧠 Amazon ElastiCache SHOULD KNOW

What it is: A fully managed in-memory data store and cache service, supporting Redis and Memcached.

Analogy: Keeping your most frequently used tools on your workbench (RAM) instead of walking to the tool shed (Database) every time.

Why and when to use it: To drastically improve application read performance by caching frequent

📊 Amazon Redshift SHOULD KNOW

What it is: A fully managed, petabyte-scale cloud data warehouse designed for complex analytics.

Analogy: A massive library optimized for researchers who need to analyze millions of books simultaneously to find trends, rather than quickly checking out a single book.

Why and when to use it: Business Intelligence (BI), reporting, and running complex analytical queries (OLAP) across massive historical datasets.

How it works: Uses columnar storage and massively parallel processing (MPP) to run queries across multiple compute nodes simultaneously.

Pricing model: Billed per hour based on the type and number of nodes in your cluster.

Category 5: Security and Identity 🔐

Security in AWS is paramount. A single misconfigured policy can expose your entire infrastructure. As a DevOps engineer, you must master IAM and secret management.

🛂 AWS IAM (Identity and Access Management) MUST KNOW

What it is: The service that controls access to AWS resources. It dictates *who* is authenticated (signed in) and *what* they are authorized to do.

Analogy: The security badge system for a highly secure corporate building. It dictates who gets a badge, and which specific rooms that badge can unlock.

Why and when to use it: Always. It is the core of AWS security.

Key Concepts (The Core Four):

- **Users:** A specific person or application (e.g., "Alice" or "CI/CD-Service-Account"). They get long-term credentials (password or Access Keys).
- **Groups:** A collection of users (e.g., "Developers"). You assign permissions to the group, and users inherit them.
- **Policies:** JSON documents that explicitly define permissions (e.g., "Allow S3 Read, Deny EC2 Delete"). Attached to Users, Groups, or Roles.
- **Roles:** Like a User, but with no permanent credentials. It is a "hat" that an entity can temporarily put on. *This is crucial for DevOps.*

How it works (Roles in action): You do NOT put AWS access keys on an EC2 instance. Instead, you create an IAM Role with S3 access, and attach that Role to the EC2 instance. The application on EC2 securely asks AWS for temporary credentials in the background.

```
[EC2 Instance] --assumes--> [IAM Role] --grants access--> [S3 Bucket]
```

🔑 AWS KMS (Key Management Service) MUST KNOW

What it is: A managed service that allows you to create and control the cryptographic keys used to protect your data.

Analogy: A highly secure master lockbox. You use the master key inside it to lock/unlock smaller boxes (your data), but you can never actually take the master key out of the facility.

Why and when to use it: Encrypting EBS volumes, S3 buckets, RDS databases, and Secrets Manager. Essential for compliance and security.

Key concepts: CMK (Customer Master Key). Symmetric vs. Asymmetric encryption. Envelope Encryption (encrypting a data key with a master key).

Pricing model: ~$1/month per key, plus a few cents per 10,000 API requests.

> ⚠️ **Warning:** If you delete a KMS key that was used to encrypt an EBS volume, that data is permanently gone forever. Proceed with extreme caution.

🤫 AWS Secrets Manager MUST KNOW

What it is: A service that helps you protect secrets needed to access your applications, services, and IT resources (like database passwords and API keys).

Analogy: A digital vault for your passwords that can automatically change the locks (rotate passwords) on a schedule.

Why and when to use it: Instead of putting database credentials in your code or environment variables, your code makes an API call to Secrets Manager at runtime to get the password.

Key concepts: Automatic Rotation (it can use a Lambda function to actually log into RDS, change the password, and update the vault automatically).

Pricing model: $0.40 per secret per month + API call costs.

Related services: RDS (native integration), Lambda, KMS, IAM.

Note: *AWS Systems Manager Parameter Store* is a cheaper/free alternative for storing simple configuration strings, but it lacks automatic native rotation.

📜 AWS Certificate Manager (ACM) SHOULD KNOW

What it is: A service that lets you easily provision, manage, and deploy public and private Secure Sockets Layer/Transport Layer Security (SSL/TLS) certificates.

🛡️ AWS WAF & Shield SHOULD KNOW

What it is: WAF is a Web Application Firewall that helps protect against common web exploits (SQL injection, XSS). Shield provides DDoS protection.

Analogy: Shield is the outer moat stopping a massive brute-force army (DDoS). WAF is the meticulous guard at the door checking if anyone is carrying a concealed weapon (SQL injection).

Why and when to use it: Attach WAF to your Application Load Balancer or CloudFront distribution to block malicious traffic patterns or rate-limit IPs.

Pricing model: WAF charges per Web ACL rule and per million requests. Shield Standard is free and automatic; Shield Advanced costs $3,000/month.

👁️ Amazon GuardDuty GOOD TO KNOW

What it is: A threat detection service that continuously monitors for malicious activity and unauthorized behavior.

Analogy: A silent security camera system running AI that alerts you if an employee suddenly starts accessing files at 3 AM from an IP address in a restricted country.

How it works: It requires zero architecture changes. You click "Enable," and it analyzes CloudTrail, VPC Flow Logs, and DNS logs in the background using Machine Learning.

👤 Amazon Cognito SHOULD KNOW

What it is: Customer Identity and Access Management (CIAM). It provides sign-up, sign-in, and access control to your web and mobile apps.

Analogy: Instead of building your own login page, user database, and "Forgot Password" flow, you rent AWS's battle-tested login system.

Why and when to use it: When you are building a B2C application and need users to log in via email, Google, Facebook, or Apple. (Note: This is for YOUR app's users, whereas IAM is for your internal DevOps/Engineers).

📝 **DevOps Practice Lab 4 (Databases):** Deploy an Amazon Aurora Serverless v2 PostgreSQL database. Do not make it public. Deploy a t3.micro EC2 bastion host in the same VPC to test connecting to the database using `psql`.

📝 **DevOps Practice Lab 5 (Security):** Create an IAM Role that has "AmazonS3ReadOnlyAccess". Create an EC2 instance and attach this role to it (Instance Profile). SSH into the instance and run `aws s3